

Event, State, dan Implisit Intent pada Jetpack Compose

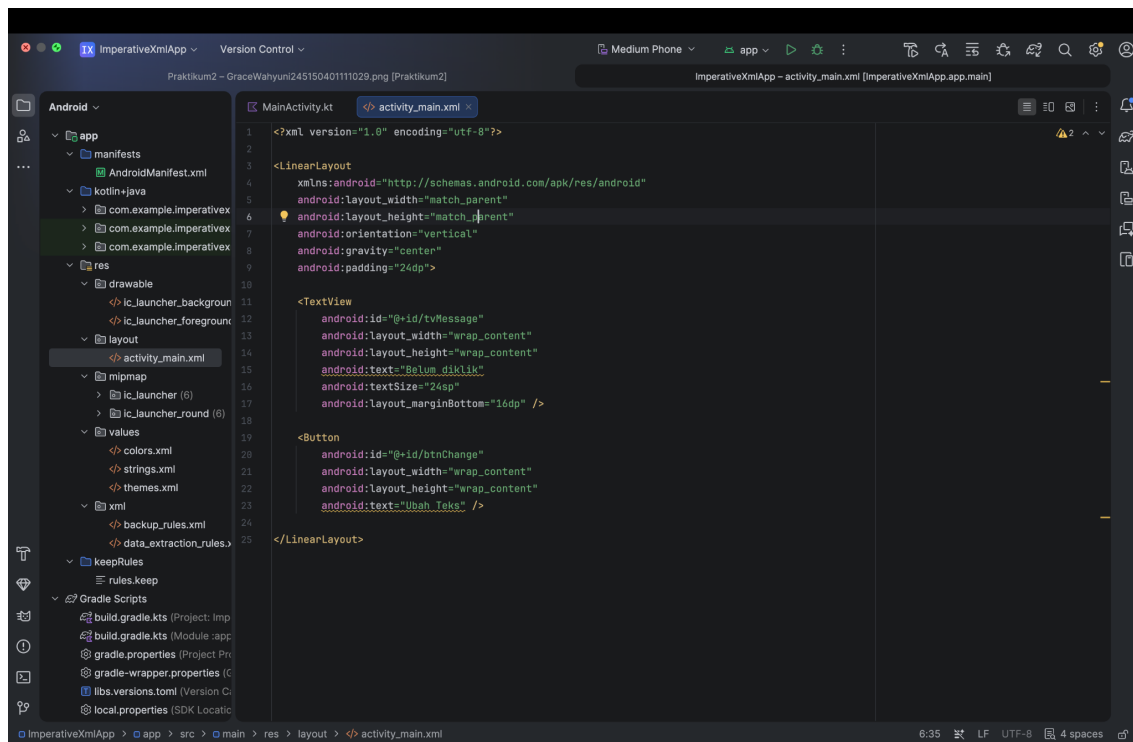
Nama	Grace Wahyuni
NIM	245150401111029
Mata Kuliah	Pengembangan Aplikasi Mobile
Dosen Pengampu	Muhammad Aminul Akbar, S.Kom., M.T.

JAWABAN

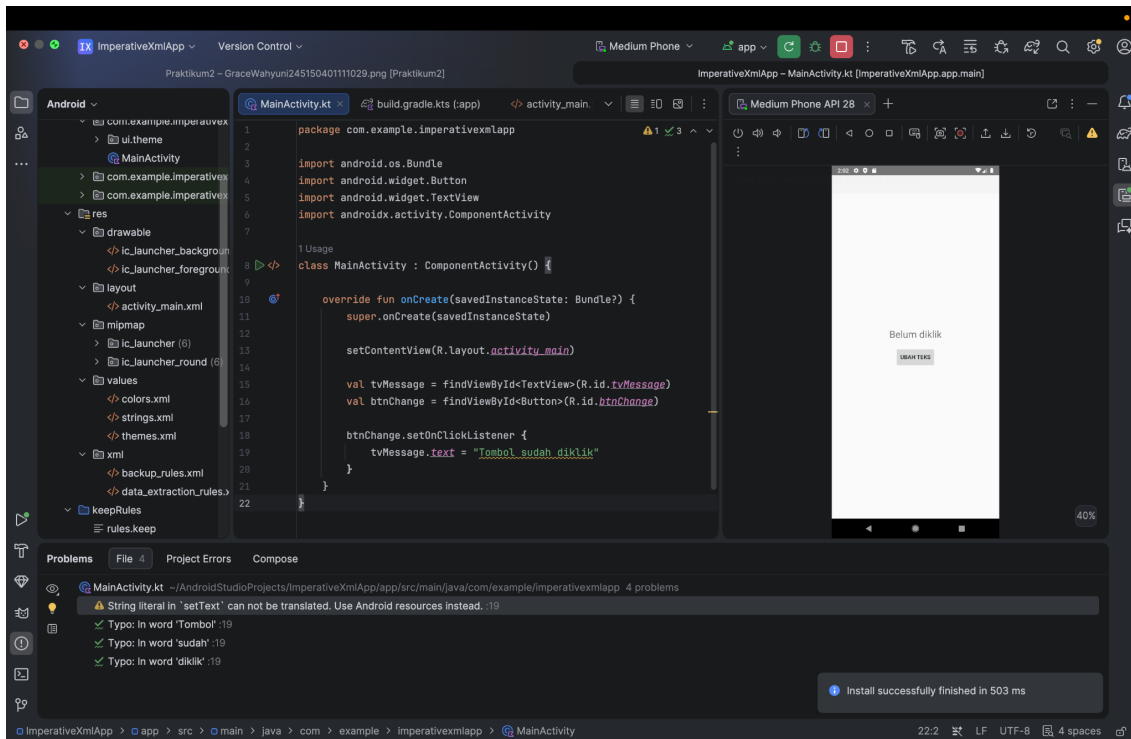
1.3 Percobaan 1: Perbandingan Imperatif dan Deklaratif

Percobaan ini membandingkan cara perubahan UI pada XML/View (imperatif) dan Jetpack Compose (deklaratif). Hasil akhir kedua aplikasi sama, tetapi mekanisme pembaruan tampilannya berbeda.

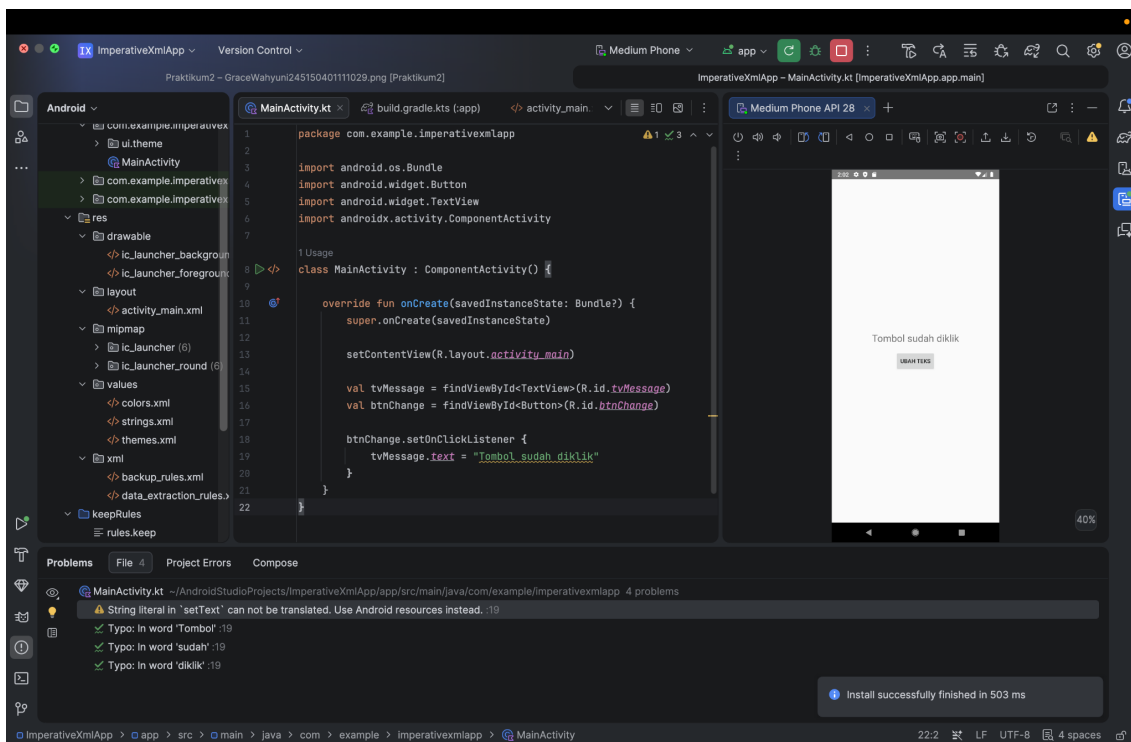
A. Project A - XML/View (Imperatif)



Gambar 1. Kode activity_main.xml pada project XML/View.

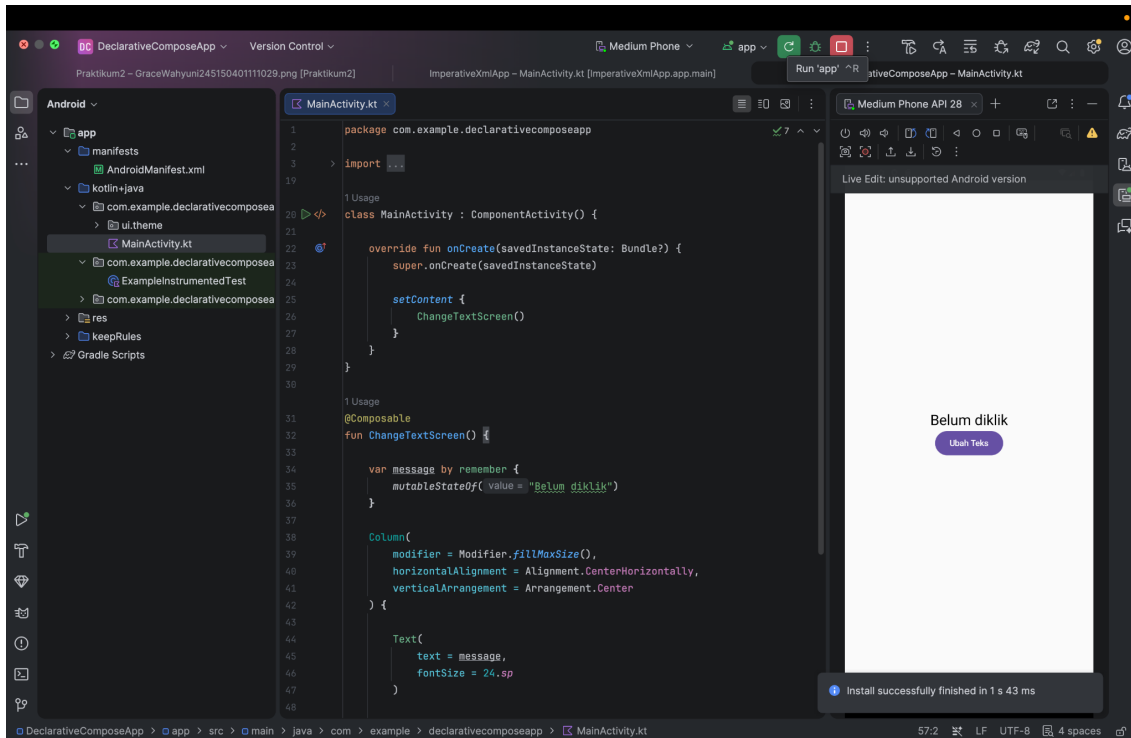


Gambar 2. Kondisi awal XML/View dengan teks "Belum diklik".

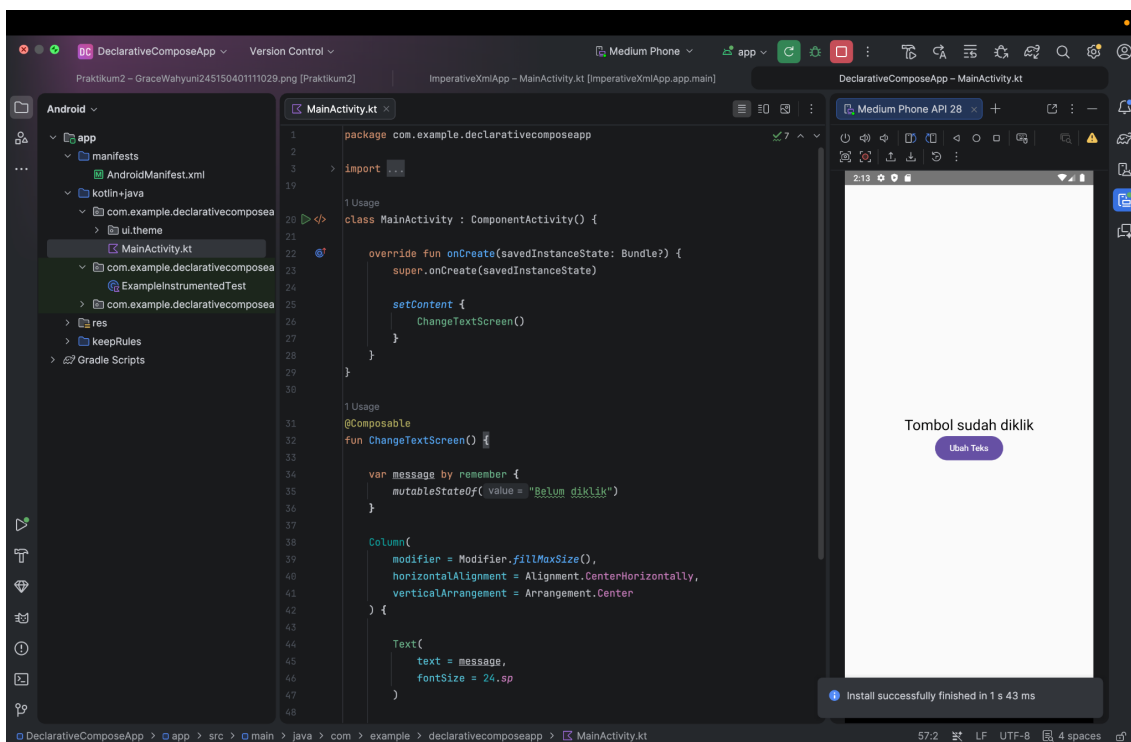


Gambar 3. Kondisi XML/View setelah tombol "Ubah Teks" ditekan.

B. Project B - Jetpack Compose (Deklaratif)



Gambar 4. Kondisi awal Jetpack Compose dengan teks "Belum diklik".



Gambar 5. Kondisi Jetpack Compose setelah tombol "Ubah Teks" ditekan.

Jawaban Aktivitas Percobaan

2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Objek yang diubah secara langsung adalah TextView bernama tvMessage. Baris yang melakukan perubahan adalah `tvMessage.text = "Tombol sudah diklik"`. Program mengambil

referensi View melalui `findViewById()`, kemudian mengubah properti text pada objek tersebut.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

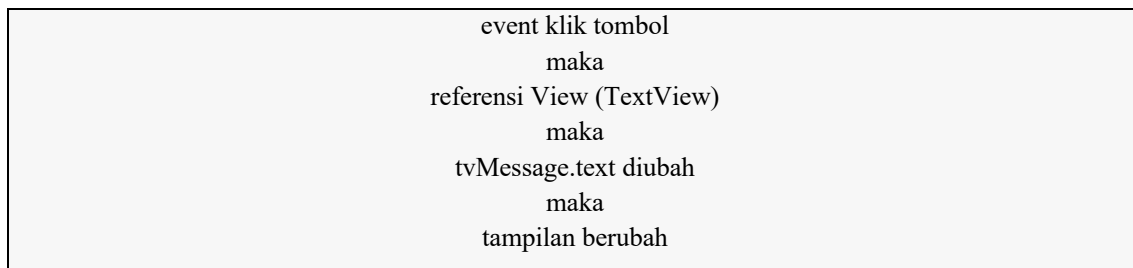
Nilai yang diubah adalah state message. Baris yang mengubahnya adalah `message = "Tombol sudah diklik"`. Komponen `Text(text = message)` hanya membaca nilai state tersebut untuk menentukan teks yang ditampilkan.

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Karena Jetpack Compose menggunakan pendekatan deklaratif. Pengembang tidak mengubah objek UI secara langsung. UI didefinisikan berdasarkan state saat ini, ketika state berubah, Compose melakukan recomposition pada bagian UI yang bergantung pada state tersebut.

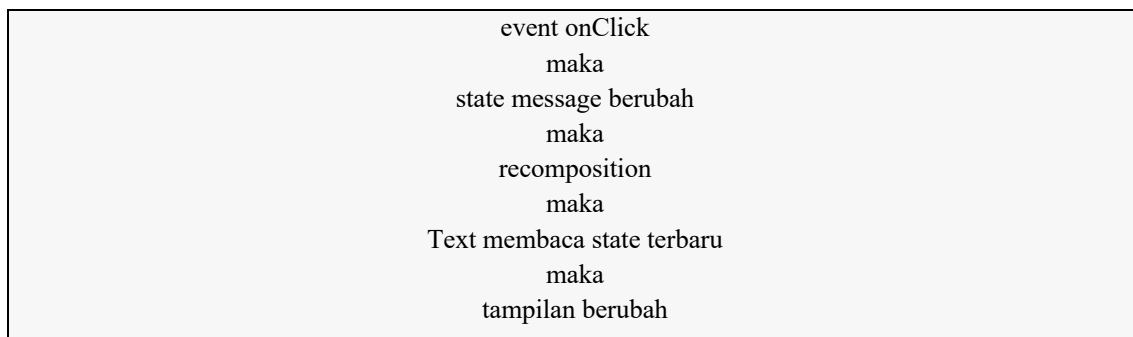
5. Jelaskan alur perubahan pada XML/View dengan pola event -> View -> perubahan tampilan.

Ketika pengguna menekan tombol, event `setOnClickListener` dijalankan. Program kemudian menggunakan referensi `TextView` dan mengubah properti text secara langsung. Setelah properti View berubah, tampilan pada layar ikut berubah.



6. Jelaskan alur perubahan pada Compose dengan pola event -> state -> recomposition -> perubahan tampilan.

Ketika pengguna menekan tombol, event `onClick` mengubah state message. Karena message merupakan state yang diamati Compose, perubahan tersebut memicu recomposition. Saat recomposition, `Text` membaca nilai message terbaru sehingga tampilan diperbarui.



7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Nilai variabel biasa dapat berubah di dalam event, tetapi perubahan tersebut tidak otomatis memicu recomposition. Compose tidak mengobservasi variabel lokal biasa sebagai state, sehingga perubahan nilainya tidak menjamin UI digambar ulang. Agar perubahan UI dapat diamati, data perlu menggunakan state seperti `mutableStateOf()`.

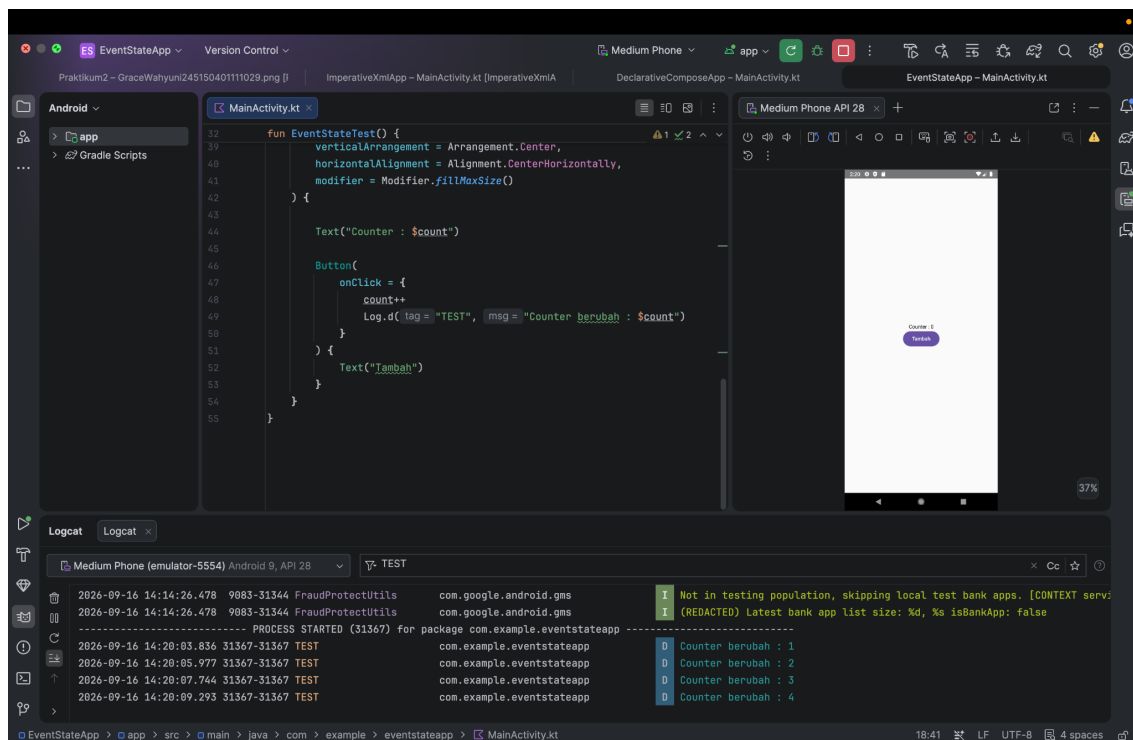
8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Pendekatan imperatif berfokus pada perintah langsung terhadap komponen UI, dimana program mencari View tertentu lalu mengubah propertinya. Pendekatan deklaratif berfokus pada state, dimana program mendeskripsikan bagaimana UI seharusnya terlihat berdasarkan state saat ini, kemudian framework memperbarui UI ketika state berubah.

1.4 Percobaan 2: Event Handling State

Percobaan ini mengamati perbedaan perilaku variabel biasa dan state pada Jetpack Compose, kemudian menambahkan event input melalui `OutlinedTextField`.

A. Percobaan dengan Variabel Biasa



Gambar 6. Hasil percobaan saat count masih berupa variabel biasa; Logcat berubah ketika tombol "Tambah" ditekan.

2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Setiap tombol "Tambah" diklik, Logcat menampilkan pesan Counter berubah : n, dengan nilai n yang bertambah sesuai jumlah klik, misalnya 1, 2, 3, dan seterusnya.

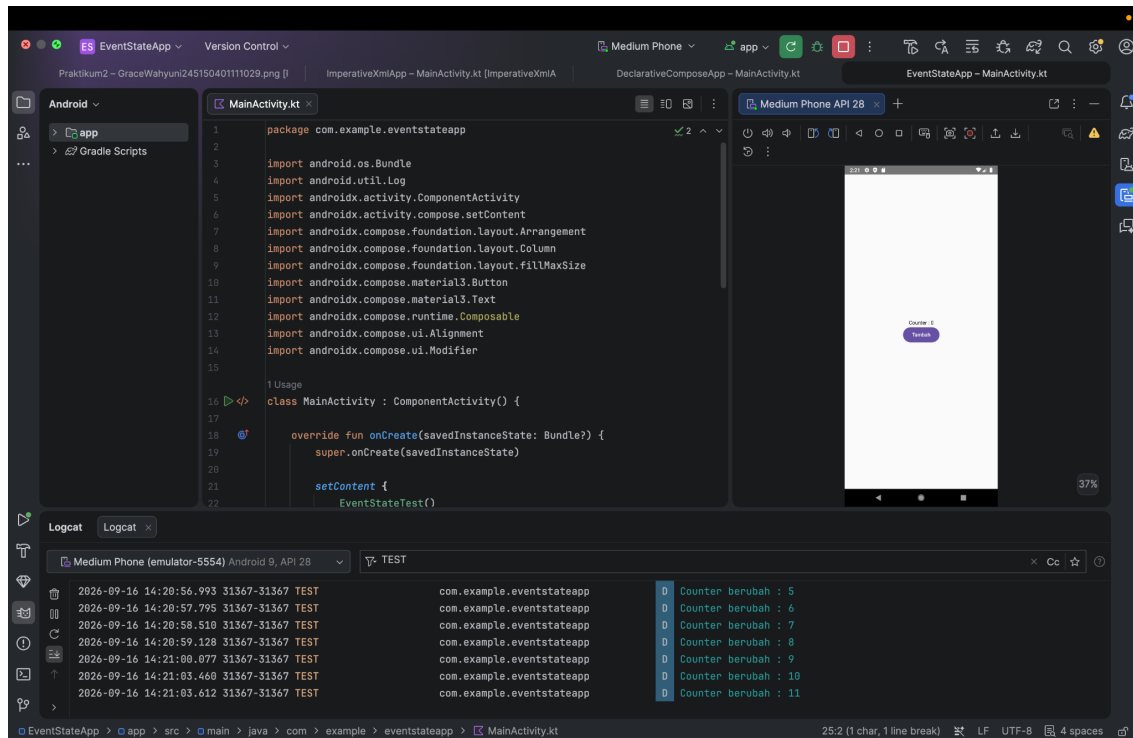
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Nilai yang terlihat pada UI tidak mengikuti perubahan yang tercatat di Logcat. Tampilan counter tetap pada nilai awal karena perubahan terjadi pada variabel biasa yang tidak diamati oleh Compose.

4. Apa yang menyebabkan hal tersebut terjadi?

Penyebabnya adalah count dideklarasikan sebagai variabel biasa (`var count = 0`). Perubahan pada variabel biasa tidak memicu recomposition, sehingga Compose tidak otomatis memperbarui komponen Text yang menampilkan counter.

B. Percobaan dengan State



Gambar 7. Hasil setelah count diubah menjadi state dengan `remember` dan `mutableStateOf`.

9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Logcat tetap menampilkan perubahan nilai counter, misalnya Counter berubah : 1, Counter berubah : 2, dan seterusnya sesuai jumlah klik.

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Nilai counter pada UI ikut bertambah setiap tombol ditekan. Teks yang menampilkan counter selalu memperlihatkan nilai state terbaru.

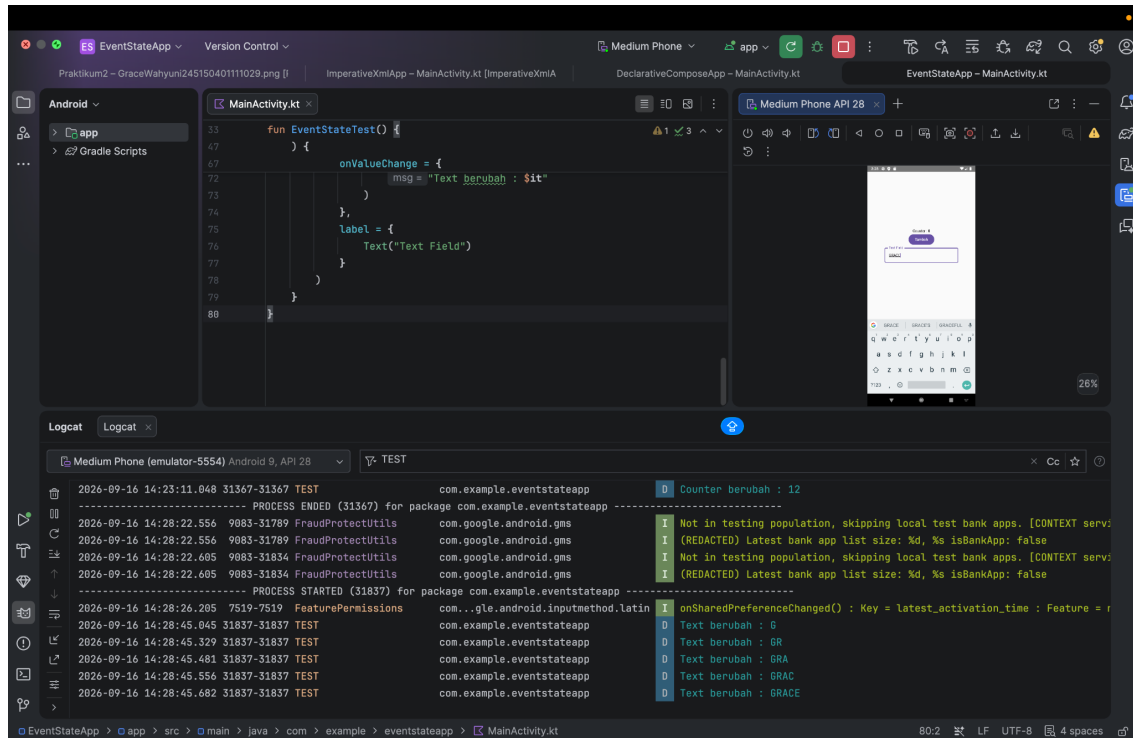
11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Pada percobaan ini count menggunakan `remember { mutableStateOf(0) }`. Nilai tersebut merupakan state yang dapat diamati Compose. Ketika nilainya berubah, Compose menjadwalkan recomposition sehingga bagian UI yang membaca count diperbarui.

12. Apa yang dimaksud recomposition pada Compose?

Recomposition adalah proses ketika Jetpack Compose menjalankan kembali bagian Composable yang bergantung pada state yang berubah. Tujuannya agar UI mencerminkan nilai state terbaru tanpa harus mengubah objek View secara manual.

C. Penambahan State Text dan Event onChange



Gambar 8. Hasil percobaan OutlinedTextField; perubahan input tampil pada UI dan tercatat pada Logcat.

15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Setiap perubahan input menghasilkan log Text berubah : Nilai yang dicetak merupakan isi Text Field terbaru, sehingga ketika mengetik sebuah kata, Logcat menampilkan perubahan nilai secara bertahap sesuai karakter yang dimasukkan.

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Karakter yang dimasukkan langsung terlihat pada OutlinedTextField. Isi Text Field selalu mengikuti nilai state text terbaru.

17. Apa yang menyebabkan hal tersebut terjadi?

Event onChange dijalankan setiap kali nilai input berubah. Di dalam event tersebut, state text diperbarui dengan text = it. Karena text merupakan state, perubahan tersebut memicu recomposition dan OutlinedTextField menampilkan nilai baru.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Terdapat dua state, yaitu count bertipe Int untuk menyimpan nilai counter dan text bertipe String untuk menyimpan isi Text Field.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

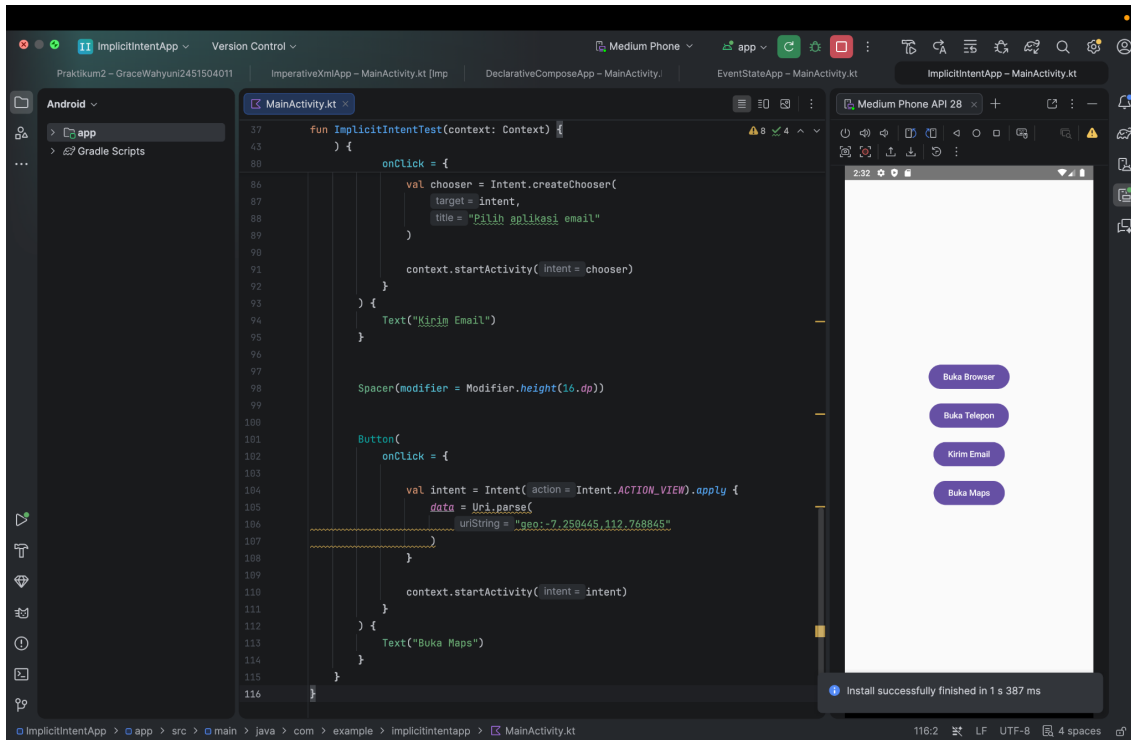
Terdapat dua event utama, yaitu onClick pada Button dan onChange pada OutlinedTextField.

20. Kapan event-event tersebut dijalankan?

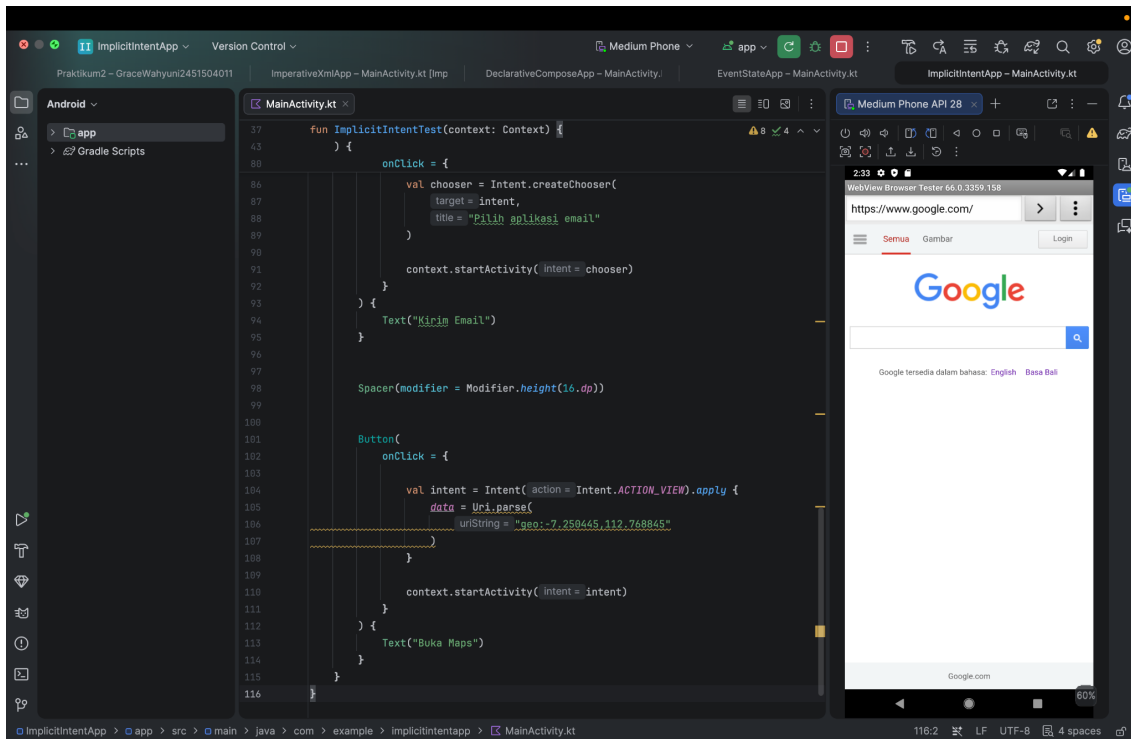
Ketika `onClick` dijalankan pengguna maka menekan tombol Tambah. `onValueChange` dijalankan setiap kali isi Text Field berubah, misalnya ketika pengguna mengetik, menghapus, atau mengganti karakter.

1.5 Percobaan 3: Implicit Intent

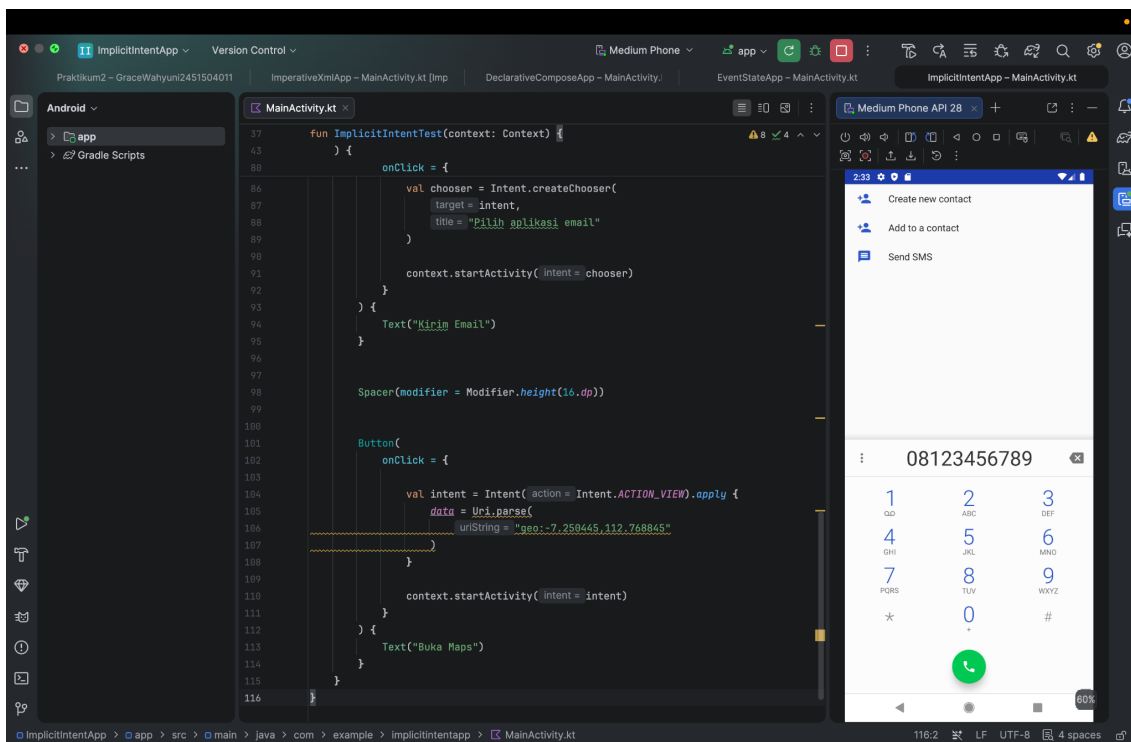
Percobaan ini menguji implicit intent untuk membuka browser, dialer telepon, aplikasi email, dan peta. Sistem Android menentukan aplikasi tujuan berdasarkan kombinasi action dan data pada Intent.



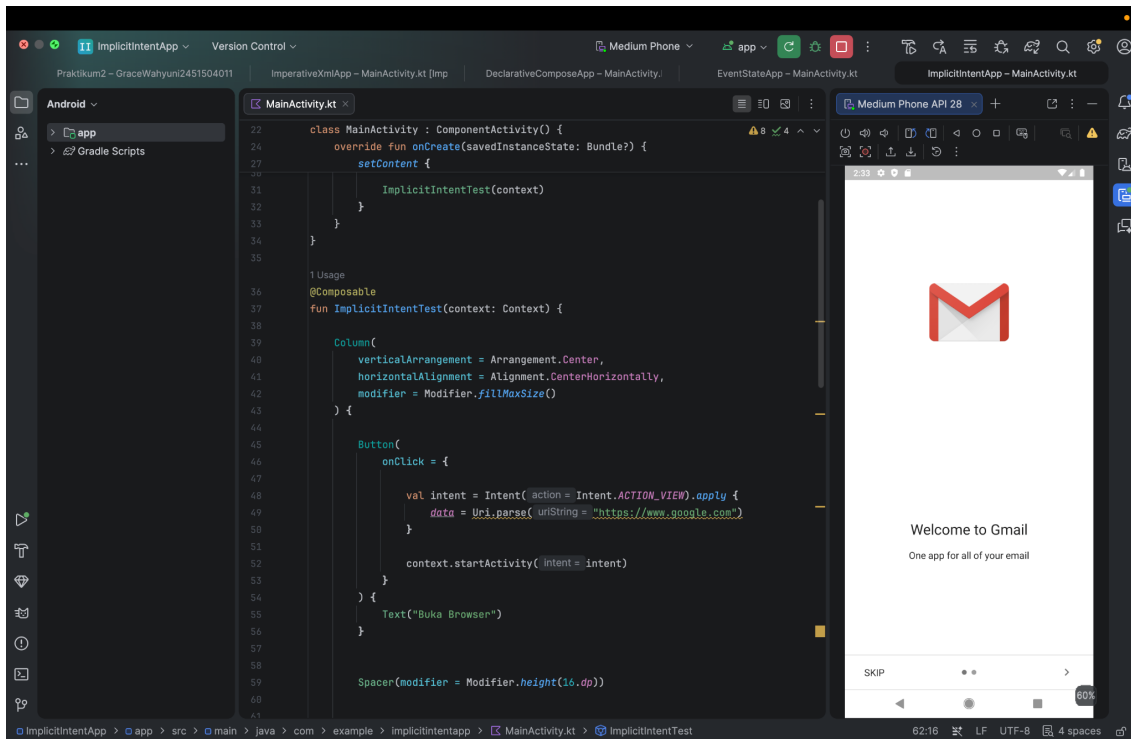
Gambar 9. Tampilan awal aplikasi dengan empat tombol implicit intent.



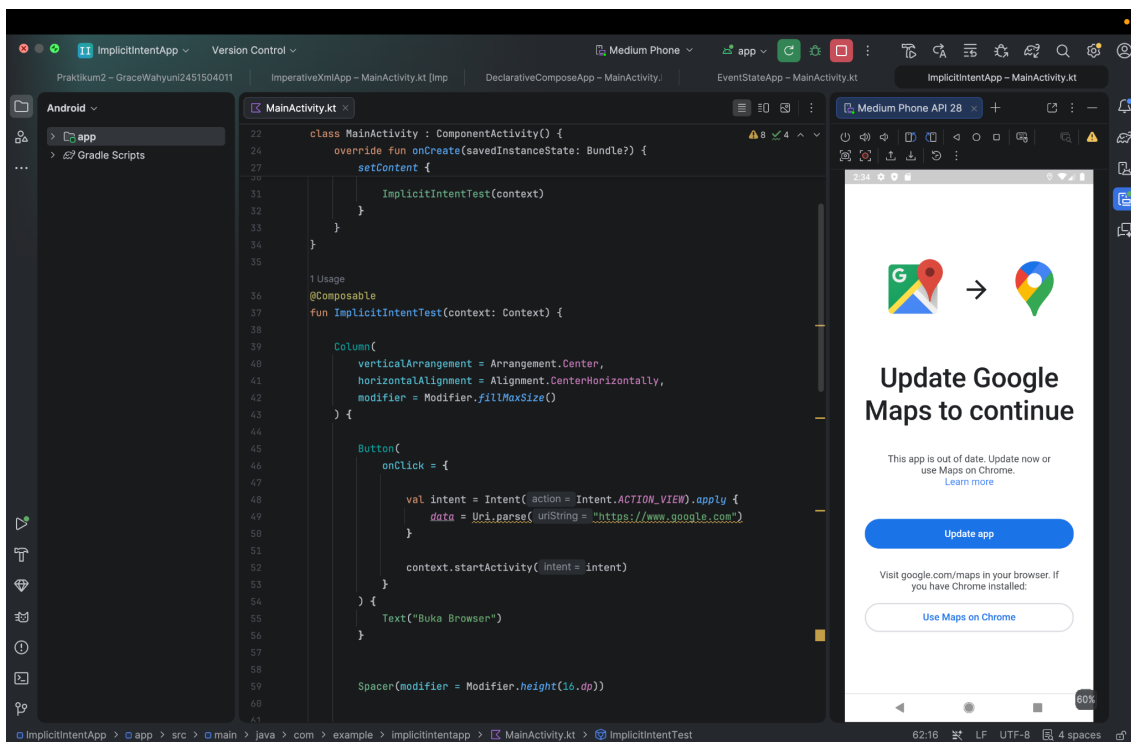
Gambar 10. Hasil tombol Buka Browser: URL Google dibuka melalui aplikasi yang sesuai.



Gambar 11. Hasil tombol Buka Telepon: dialer terbuka dengan nomor tujuan.



Gambar 12. Hasil tombol Kirim Email: aplikasi/chooser email dibuka.



Gambar 13. Hasil tombol Buka Maps pada emulator.

Jawaban Aktivitas Percobaan

11. Apa perbedaan fungsi dari Intent.ACTION_VIEW, Intent.ACTION_DIAL, dan Intent.ACTION_SENDTO?

Intent.ACTION_VIEW digunakan untuk meminta sistem menampilkan atau membuka suatu data, misalnya URL web atau lokasi geografis. Intent.ACTION_DIAL digunakan untuk membuka aplikasi dialer dengan nomor telepon yang sudah terisi tanpa langsung

melakukan panggilan. Kemudian, `Intent.ACTION_SENDTO` digunakan untuk mengirim data ke tujuan tertentu, pada percobaan ini dipadukan dengan URI `mailto:` sehingga sistem mencari aplikasi email yang sesuai.

16. Apa fungsi data pada intent?

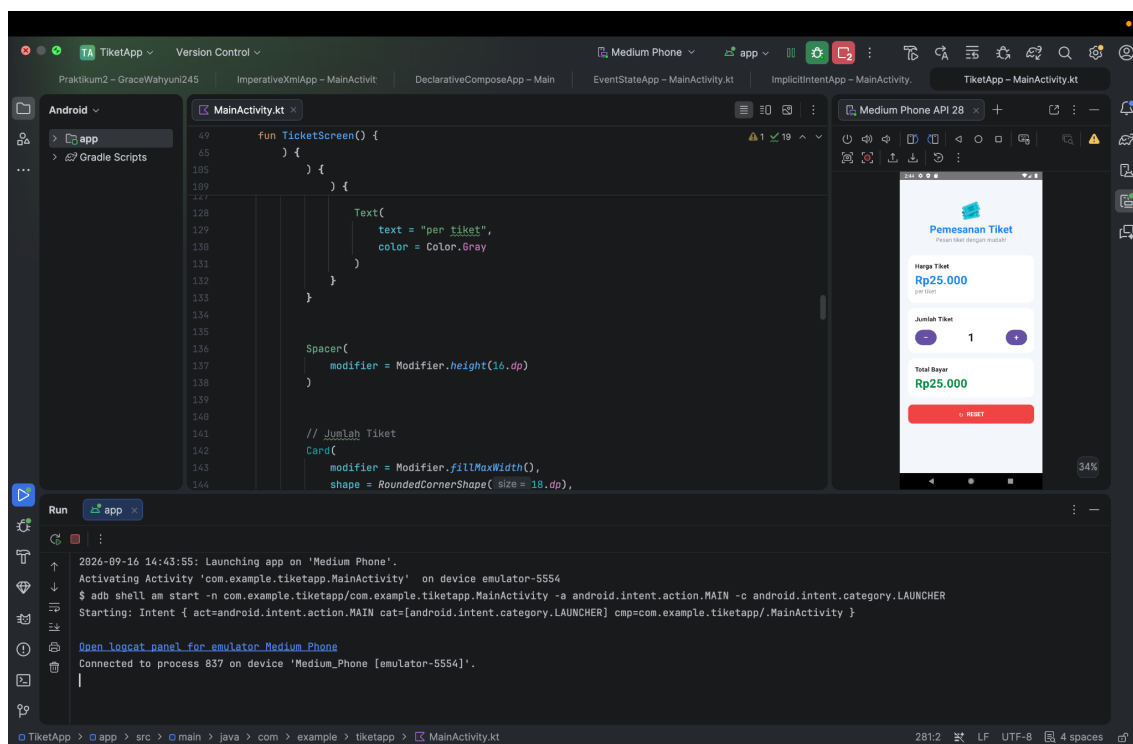
Data pada Intent memberikan informasi tentang objek atau tujuan yang akan diproses. Contohnya, `https://www.google.com` menunjukkan alamat web, `mailto:test@gmail.com` menunjukkan alamat email tujuan, `tel:08123456789` menunjukkan nomor telepon, dan `geo:-7.250445,112.768845` menunjukkan lokasi geografis.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

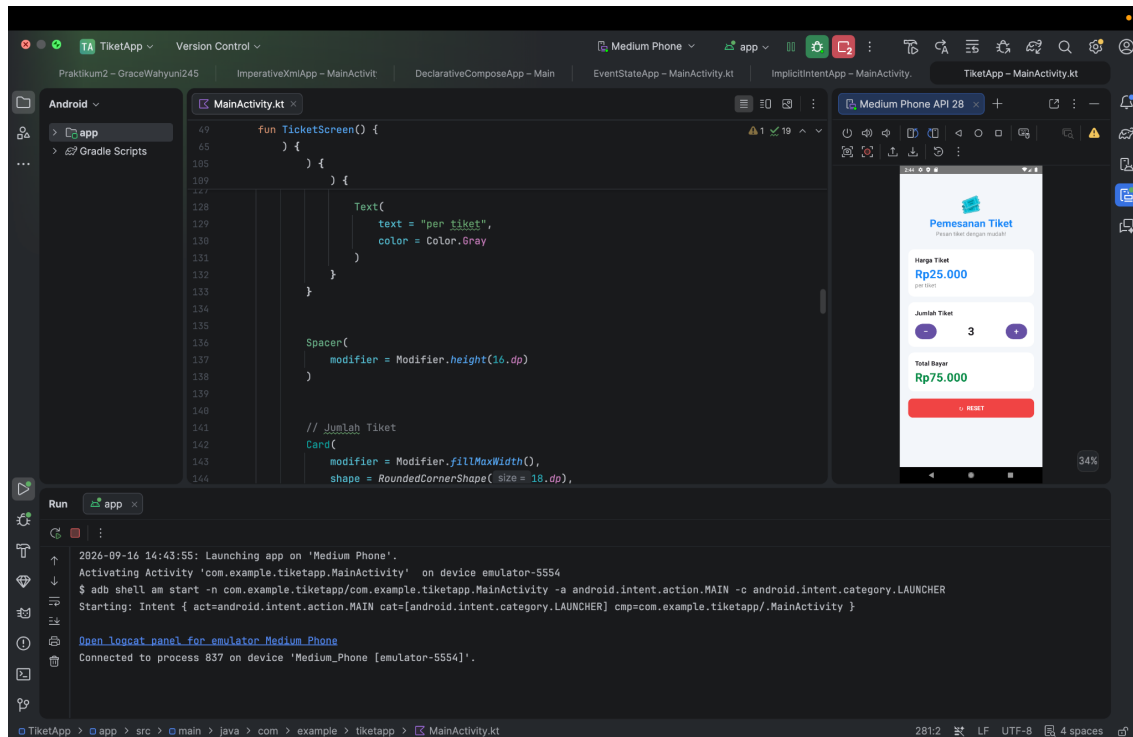
Android menggunakan kombinasi Action dan Data untuk mencocokkan Intent dengan aplikasi yang memiliki intent-filter yang sesuai. Misalnya `ACTION_VIEW` dengan data berformat `https` dapat ditangani browser, `ACTION_DIAL` dengan data `tel` ditangani dialer, `ACTION_SENDTO` dengan data `mailto` ditangani aplikasi email, dan `ACTION_VIEW` dengan data `geo` dapat ditangani aplikasi peta. Karena menggunakan implicit intent, aplikasi tujuan tidak ditentukan secara eksplisit oleh kode.

1.6 Tugas Praktik: Pemesanan Tiket

Tugas praktik membuat halaman pemesanan tiket yang menampilkan harga tiket, jumlah tiket yang dapat diubah, dan total bayar. Harga tiket ditetapkan Rp25.000 per tiket. Jumlah tiket disimpan sebagai state sehingga setiap perubahan jumlah tiket otomatis memperbarui total bayar.



Gambar 14. Kondisi awal pemesanan: 1 tiket dengan total Rp25.000.



Gambar 15. Setelah tombol + ditekan dua kali: 3 tiket dengan total Rp75.000.

Analisis Implementasi

State jumlah tiket: Jumlah tiket disimpan menggunakan state (remember dan mutableStateOf) sehingga perubahan nilai dapat diamati oleh Compose.

Event tombol + dan - : Tombol + menambah jumlah tiket, sedangkan tombol - mengurangi jumlah tiket dengan batas minimum 1 tiket.

Perhitungan total: Total bayar diperoleh dari $\text{hargaTiket} \times \text{jumlahTiket}$. Karena perhitungan bergantung pada state jumlah tiket, nilai total akan dihitung kembali saat terjadi recomposition.

Reset: Tombol RESET mengembalikan jumlah tiket ke nilai awal 1 sehingga total kembali menjadi Rp25.000.

Kesimpulan

Melalui praktikum ini, perbedaan pendekatan imperatif dan deklaratif dapat diamati secara langsung. Pada XML/View, komponen UI diubah melalui referensi objek View, sedangkan pada Jetpack Compose tampilan mengikuti state. State yang dibuat dengan remember dan mutableStateOf memungkinkan perubahan data memicu recomposition. Event seperti onClick dan onValueChange digunakan untuk mengubah state, sementara implicit intent memungkinkan aplikasi meminta sistem membuka aplikasi lain berdasarkan action dan data. Konsep event dan state tersebut kemudian diterapkan pada halaman pemesanan tiket agar jumlah tiket dan total bayar dapat berubah secara dinamis.